Sukkur Institute of Business Administration University

Department of Computer Science

Object Oriented Programming using Java

BS – II (CS/AI/SE)

Spring-2025

Lab # 10: Let's learn about Packages and Interfaces(Abstraction)

Instructor: Nimra Mughal

Objectives

After performing this lab, students will be able to understand:

- Packages
- Interfaces basics

Package

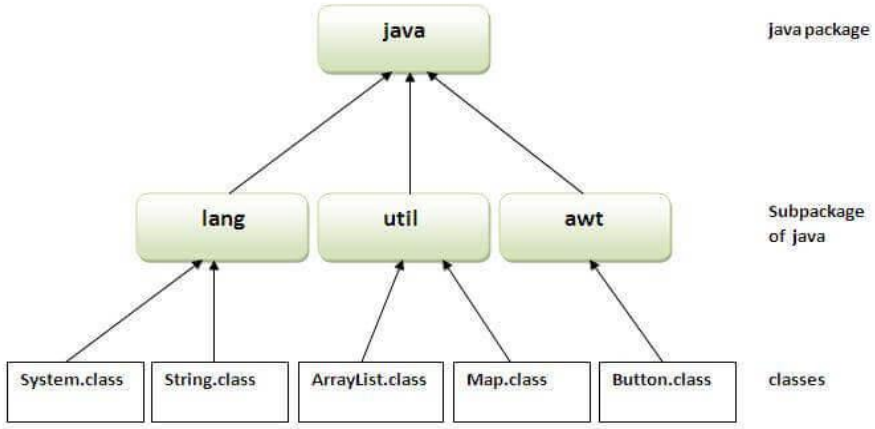
A **package** is a namespace that organizes a set of related classes and interfaces. Conceptually you can think of packages as being similar to different folders on your computer.

Package in java can be categorized in two form, built-in package and user-defined package.

There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.

Advantages of using Java Packages:

1. **Better organization**: As in large java projects where we have several hundreds of classes, it is always required to group the similar types of classes in a meaningful package name.
2. **Reusability**: While developing a project in java, we often feel that there are few things that we are writing again and again in our code. Using packages, you can create such things in form of classes inside a package and whenever you need to perform that same task, just import that package and use the class.
3. **Name conflicts**: Java package removes naming collision. For example; We can define two classes with the same name in different packages so to avoid name collision, we can use packages



Types of packages in Java

- *User defined package*: The package we create is called user-defined package.
- *Built-in package*: The already defined package like java.io.*, java.lang.* etc are known as built-in packages.

Example

A class Calculator is created inside a package name letmecalculate. To create a class inside a package, declare the package name in the first statement in your program.

A class can have only one package declaration.

Calculator.java file created inside a package letmecalculate

```
package letmecalculate;

public class Calculator {
   public int add(int a, int b){
       return a+b;
   }
}// save this file as Calculator.java. This class is used to create a package as
well as to perform addition
```

Now lets see how to use this package in another program.

```
import letmecalculate.Calculator;

public class Demo{
   public static void main(String args[]){
       Calculator obj = new Calculator();
       System.out.println(obj.add(100, 200));
   }
}// save this file as Demo.java
```

Steps for compiling and running java package

1. Access path of the folder/directory where you have saved the Calculator file
   For example : C:\Users\HP\Desktop\OOP, it suggests that Calculator file is on desktop in OOP folder
2. javac -d . Calculator.java \\ compile the package
3. javac Demo.java
4. java Demo

Access package from another package

There are three ways to access the package from outside the package.

1. Import package.classname;
2. Fully qualified name.
3. Import package.*

1. Using package.classname

```
import letmecalculate.Calculator;
```

2. Using fully qualified name

```
public class Demo{
```

```
   public static void main(String args[]){
    letmecalculate.Calculator obj = new letmecalculate.Calculator();
    System.out.println(obj.add(100, 200));
   }
}
```

3. **Using package.***

   If you use package.* then all the classes and interfaces of this package will be accessible but not subpackages.

```
Import java.util.*;
```

Interfaces

An interface in Java is a blueprint of a class. The interface in Java is a mechanism to achieve abstraction. There can be only abstract methods in the Java interface, not method body. It is used to achieve abstraction and multiple inheritance in Java.

Since Java 8, we can have **default** and **static methods** in an interface.

Since Java 9, we can have **private methods** in an interface.

Why interfaces?

These are reasons to use interface:

- It is used to achieve abstraction.
- By interface, we can support the functionality of multiple inheritance.

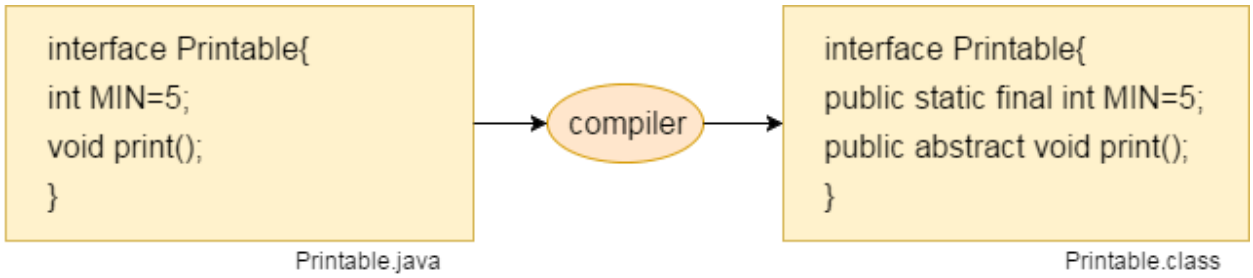
Declaration of an Interface

An interface is declared by using the interface keyword. It provides total abstraction; means all the methods in an interface are declared with the empty body, and all the fields are public, static and final by default. A class that implements an interface must implement all the methods declared in the interface.

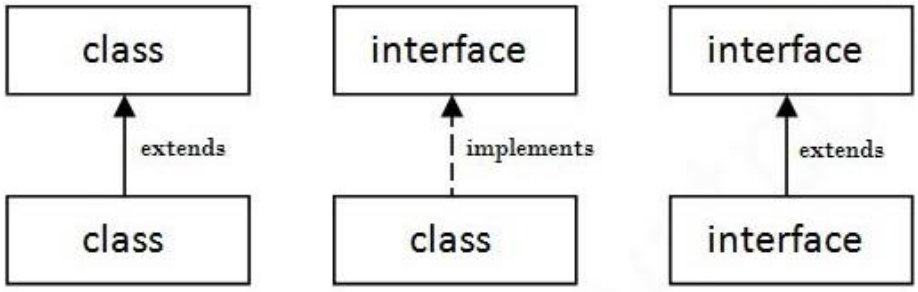
Syntax:

```
interface <interface_name>{
    // declare constant fields
    // declare methods that abstract
    // by default.
}
```

Interface fields are public, static and final by default, and the methods are public and abstract.



The relationship between classes and interfaces



Example:

Open interface_VehicleExample.java

Default Method in Interface

Since Java 8, we can have method body in interface. But we need to make it default method. Let's see an example:

```
interface Drawable{
     void draw();
     default void msg(){System.out.println("default method");}
}
class Rectangle implements Drawable{
     public void draw(){System.out.println("drawing rectangle");}
}
class TestInterfaceDefault{
     public static void main(String args[]){
          Drawable d=new Rectangle();
          d.draw();
          d.msg();
}}
```

Static Method in Interface

```
interface Drawable{
     void draw();
     static int cube(int x){return x*x*x;}
}
class Rectangle implements Drawable{
      public void draw(){System.out.println("drawing rectangle");}
}
class TestInterfaceStatic{
      public static void main(String args[]){
          Drawable d=new Rectangle();
          d.draw();
          System.out.println(Drawable.cube(3));
      }
}
```

Exercises

Question: 1

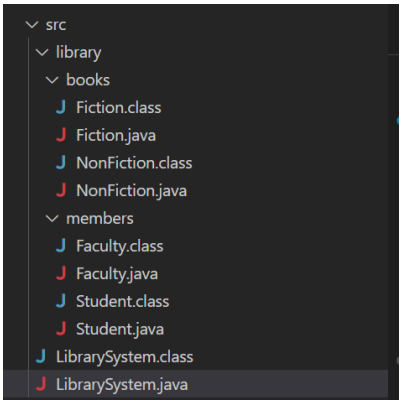
1. Create three packages, think of them yourself
2. Put two different classes in each package
3. Import all three of these packages in class named PackagePractice, you will have access to 6 classes
4. Call methods of these 6 classes and use them in PackagePractice

Refer Elearning for solution Example

Question: 2 (Packages)

1. Create a package named library.books.
    - Add classes Fiction and NonFiction, both having a displayInfo() method.
2. Create another package named library.members.
    - Add classes Student and Faculty with methods like borrowBook() and returnBook().
3. Create a class LibrarySystem in main package and:
    - Import all classes.
    - Call methods and simulate borrowing/returning books.

Sample project hierarchy structure and output




```
Fiction Book: 'The Alchemist' by Paulo Coelho.
Non-Fiction Book: 'Sapiens: A Brief History of Humankind' by Yuval Noah Harari.

Student borrowed a Fiction book.
Student returned the book.

Faculty borrowed a Non-Fiction book.
Faculty returned the book.
```

Question: 3 (Interfaces)

What is wrong with the following interface?

```
public interface SomethingIsWrong {
    void aMethod(int aValue){
        System.out.println("Hi Mom");
    }
}
```

1. Fix the interface in question.
2. Is the following interface valid?

```
public interface Marker {
}
```

Question: 3 (Interfaces)

1. Create the Animal interface.
   a. Declare method legs.
   b. Declare a method eat.
2. Create the Spider, Caterpillar and Cat class that implements animal interface.
   a. All classes implement the Animal interface.
   b. Implement the eat and legs method according to the kind of animal (can only provide a printing statement, up to you!)
   c. Provide main method and demonstrate each of above methods.

Sample output

```
== Spider ==
Spider has 8 legs.
Spider eats insects.

== Caterpillar ==
Caterpillar has many tiny legs.
Caterpillar munches on green leaves.

== Cat ==
Cat has 4 legs.
Cat eats meat and drinks milk.
PS H:\My Drive\Collab-Learning\SIBA_Regular\Spri
```

Question: 4 (Interfaces)

1. Create an interface Shape with methods:
   - double getArea()
   - double getPerimeter()
2. Implement it in:
   - Circle
   - Rectangle
   - Triangle
3. In the main method, calculate and display area and perimeter for each shape.

Sample output

```
Circle:
Area: 78.53981633974483
Perimeter: 31.41592653589793

Rectangle:
Area: 24.0
Perimeter: 20.0

Triangle:
Area: 6.0
Perimeter: 12.0
```